Code Review And Quality

§ . . – Dead/unwired-code investigate-before-remove mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-04):

> "Before removing any seemingly-dead (zero-importer / unwired) codebase, we MUST investigate via git history where/how it was originally used and how it became dead. Removal is permitted ONLY when we have captured PROOF it is genuinely no longer needed — and that removal MUST be its own separate commit with a proper descriptive message. If there is no such proof, the code MUST be investigated for where/how it should be wired in properly, and any missing or unwired tests MUST be added. We MUST ALWAYS be extra careful with any codebase removal."

The defect class. "This has zero importers / is never called / is unwired, so it is dead — delete it" is a guess (§11.4.6), not a finding. A `git grep` / static-analysis "no references" result proves only *current* non-reference; it does NOT prove the code is genuinely unneeded. Code becomes apparently-dead for several distinct reasons — its call-site was deleted by mistake (a regression: the code SHOULD still be wired in), its wiring was never finished (it is intended but unwired — the fix is to WIRE it, plus its tests), it is reflectively / dynamically / build-tag / plugin / DI-container referenced (the static tool simply cannot see the reference), or it is genuinely obsolete and safe to remove. Deleting it without first distinguishing which case applies risks silently removing a needed-but-temporarily-unwired capability — the same severity class as the §11.4.122 silent-removal defect and a §11.4 PASS-bluff at the maintenance layer.

The mandate. Before removing ANY seemingly-dead codebase element (a zero-importer / never-called / unwired / unreferenced function, method, type, file, module, package, asset, config, or build target), the agent MUST FIRST investigate via git history (`git log --follow` , `git log -S<symbol>` / `-G<regex>` pickaxe across all history, `git log --diff-filter` , blame on the deletion of the former call-site) to establish, as captured FACT (§11.4.6): (1) WHERE and HOW it was originally used / wired in; (2) WHEN and HOW it became dead — its call-site removed deliberately, removed by mistake (regression), never completed, or made unreachable by a refactor; (3) whether the "no references" result is real OR an artefact of a reference the static tool cannot see (reflection / dynamic dispatch / build-tags / codegen / DI / plugin registry / cross-language FFI / config-driven wiring). The investigation output (the cited commits + the determination) is the captured evidence.

Removal is conditional — proof-of-genuinely-unneeded REQUIRED. Removal is permitted ONLY when the investigation produces captured PROOF the element is genuinely no longer needed (its purpose is fully superseded, its former caller is intentionally and permanently gone, no hidden reference exists). When that proof exists, the removal MUST be its OWN SEPARATE COMMIT (not bundled with unrelated work — so it is independently reviewable and revertible, composing with §11.4.84 working-tree quiescence + §11.4.92 multi-pass) carrying a proper descriptive message that cites the git-history investigation evidence (the commits that wired it in, the commit that orphaned it, the determination that it is unneeded) and — where the element is an end-user-facing capability — the §11.4.122 operator-confirmation. The §11.4.90 tracked path applies: classify the tracker item `Obsolete (→ Fixed.md)` with the appropriate reason.

No proof ⇒ wire it in + add the missing tests (do NOT delete). If the investigation does NOT yield proof the element is unneeded, the element MUST NOT be removed. Instead it MUST be investigated for WHERE and HOW it should be wired in properly (restore the deleted call-site if its removal was a regression per §11.4.114; complete the intended wiring if it was never finished), AND any missing or unwired tests for it MUST be added (the absence of a test is itself part of why it drifted into apparent-deadness — composes §11.4.27 100%-test-type-coverage + §11.4.43 TDD-fix + §11.4.115 RED-baseline). Apparent-deadness with no removal-proof is a wiring/coverage GAP to close, never a delete-on-sight.

Extra-caution default. The agent MUST ALWAYS be extra careful with any codebase removal: when uncertain whether removal-proof is sufficient, default to NOT removing (investigate + wire + test), per §11.4.6 no-guessing + §11.4.101 (an irreversible-ish, possibly-needed removal whose safety cannot be determined from evidence is exactly the class that blocks to the operator rather than being autonomously deleted) + §11.4.122 (removal of a live end-user capability requires operator confirmation). "Probably dead" is never sufficient; the bar is captured proof.

Composition. §11.4.124 composes with §11.4.6 (no-guessing — "zero importers therefore dead" is the exact guess forbidden; the determination must be FACT from git history), §11.4.8 (deep-research — git history IS the primary research corpus for how the code was used + how it died; external research applies when the wiring mechanism is unclear), §11.4.84 / §11.4.92 (the separate-removal-commit discipline — quiescent tree, multi-pass evaluation, independently revertible), §11.4.90 (Obsolete classification is the tracked path for a proven removal), §11.4.101 (autonomous-decision-over-blocking — a removal whose safety cannot be proven blocks rather than proceeds), §11.4.114 (last-known-good-tag isolation — a call-site deleted by mistake is a regression to repair, the same diagnostic the "became dead by mistake" case demands), §11.4.122 (no-silent-removal — §11.4.124 is its complement: §11.4.122 governs removal of KNOWN-shipped capabilities by operator confirmation, §11.4.124 governs removal of SEEMINGLY-dead code by git-history proof; an end-user-facing seemingly-dead element triggers BOTH), §11.4.27 / §11.4.43 / §11.4.115 (the missing-tests obligation when the element is wired in rather than removed).

Enforcement. Propagation gate `CM-COVENANT-114-124-PROPAGATION` enforces the literal anchor `11.4.124` across the consumer fleet (every CLAUDE.md / AGENTS.md / QWEN.md). Recommended gate `CM-DEAD-CODE-INVESTIGATE-BEFORE-REMOVE` (a commit-audit detector: any commit whose diff is a NET DELETION of a code element — function/type/file/module/package/asset/build-target — MUST be a removal-only commit whose message cites the git-history investigation evidence, OR the deletion is part of a tracked `Obsolete (→ Fixed.md)` item per §11.4.90; a net-deletion commit bundled with unrelated changes OR lacking the investigation citation FAILs). Paired §1.1 meta-test mutations: strip the `11.4.124` literal → propagation gate FAILs; introduce a net-

deletion commit with no investigation citation and not part of an Obsolete item → `CM-DEAD-CODE-INVESTIGATE-BEFORE-REMOVE` FAILs. (Gate-code implementation lands as a separate follow-up work item; this anchor defines the contract.)

Classification: universal (§11.4.17) — "investigate git history + capture proof-of-unneeded before removing seemingly-dead code, otherwise wire it in and add its tests; removal is its own descriptive commit" is a platform-neutral maintenance discipline reusable by ANY codebase; the consuming project supplies its concrete static-analysis / importer-graph tooling and its hidden-reference mechanisms (reflection, DI, build-tags, codegen) per §11.4.35.

Canonical authority: this Constitution.md §11.4.124 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--zero-importers-means-dead`, `--delete-unwired-on-sight`, `--skip-git-history-investigation`, `--remove-without-proof`, `--bundle-removal-with-other-work` flag exists.

2026 06 04

§ . . — Code-review-agent gate before pre-build + main build (mandatory multi-layer review) (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-04):

> "After all fixes/changes/implementations are done, BEFORE running pre-build tests and the main build, dispatch code-review agent(s) that analyze all work done + all existing data/facts + the existing codebase + current git history to determine quality, safety, and whether the fixes/changes will REALLY work; they MUST validate and verify that every test covering the fixes/changes genuinely validates the work with NO chance of false results or bluff of any kind. Any finding MUST be fixed, polished, improved, and covered with additional tests before the build proceeds. Multiple strong layers of checks."

The defect class. Running the pre-build gate sweep and the main (artifact) build directly after a batch of fixes/changes — without an independent review pass first — lets a defective fix, an error-prone change, a bluff test, or a test that does not genuinely validate the work-under-test consume an expensive build cycle and (worse) ship green. Pre-build gates and the build prove the codebase is internally consistent + buildable; they do NOT prove the work is correct, safe, will-really-work for the end user, or that the tests covering it actually catch its negation. That assurance is an independent-review responsibility that MUST run BEFORE the build, not after.

The mandate. After all fixes / changes / implementations in a batch are done, and BEFORE running the pre-build test sweep and the main build (for ANY project), the agent MUST dispatch one or more dedicated code-review agent(s) (subagent-driven by default per §11.4.70 / §11.4.20) that perform a multi-layer review. The review MUST: (1) **analyze ALL work done in the batch** — every fix / change / implementation, its source diff, and its stated intent; (2) **analyze ALL existing data + facts** — captured evidence per §11.4.5 / §11.4.69 / §11.4.107, tracker entries, prior findings, the §11.4.108 runtime-signature registry; (3) **analyze the existing codebase** — the blast radius of each change (§11.4.92 multi-pass), cross-feature interaction, contract integrity of every dependency; (4) **analyze current git history** — what each change touched, how it composes with concurrent / recent work, whether it reproduces a known-broken pattern (§11.4.114 / §11.4.124 git-history discipline); (5) **determine quality, safety, and will-it-REALLY-work** — assess whether each change is robust + not error-prone (will not solve A while creating B), safe (no host-safety / data-safety / security regression), and genuinely delivers the end-user-visible behaviour it claims (§11.4 / §107); (6) **validate + verify the tests covering the work** — confirm every test that covers a fix/change genuinely exercises the work-under-test and catches its negation, with ZERO chance of a false result or bluff of any kind (a test that PASSes on broken-for-the-user work, a metadata-only / config-only / absence-of-error / grep-without-runtime assertion, or a gate whose paired §1.1 mutation does not make it FAIL is a finding the review MUST surface per §11.4.1 / §11.4.6 / §11.4.107 / §1.1).

Findings block the build. Any finding the review surfaces — a defect, an error-prone change, a safety risk, a will-not-really-work conclusion, a bluff/false-result-capable test, a missing-coverage gap — MUST be fixed, polished, improved, and covered with additional tests (four-layer per §11.4.4(b), TDD-RED-first per §11.4.43 / §11.4.115) BEFORE the pre-build sweep and main build proceed. The

review MUST iterate (re-review after each remediation) until it surfaces no remaining blocking findings; only then does the build proceed. The review is itself anti-bluff: its conclusions are captured evidence (§11.4.5 / §11.4.69), and a review that rubber-stamps a defective batch is the same severity class as a PASS-bluff.

Multiple strong layers. The review is one of multiple strong layers of checks, complementing — never replacing — the pre-build gate sweep (§1 four-layer), the §11.4.92 multi-pass change-evaluation, the §11.4.108 four-layer fix-verification, the §11.4.110 pre-build build-readiness verdict + clash detector, and the post-build / runtime-on-clean-target / user-visible layers. §11.4.125 sits at the seam between "all batch work done" and "start the pre-build + build": it is the independent-review gate that the other layers do not provide (the multi-pass per §11.4.92 is self-review by the change author; §11.4.125 adds the structurally-separated reviewer seam per §11.4.70).

Composition. §11.4.125 composes with §11.4 (the end-user-quality covenant the review enforces), §11.4.1 (FAIL-bluffs equally caught), §11.4.4 (four-layer coverage of every remediation the review demands), §11.4.6 (no-guessing — review conclusions are FACT from captured evidence, never "looks fine"), §11.4.40 (the full-suite retest is a later gate; §11.4.125 is the pre-build review gate that precedes it), §11.4.43 (TDD-fix — remediation lands RED-first), §11.4.50 (deterministic-consistency of the tests the review validates), §11.4.70 / §11.4.20 (the review IS a subagent dispatch — structurally-separated reviewer eliminates self-review blind spots), §11.4.92 (multi-pass change-evaluation is the author-side pass; §11.4.125 is the independent-reviewer pass), §11.4.102 (any defect the review finds triggers systematic-debugging before remediation), §11.4.107 (the review verifies AV/test-validation analyzers are not themselves bluffs), §11.4.108 (the review checks each fix's runtime signature is declared + verifiable), §11.4.110 (the build-readiness verdict + clash detector are mechanical gates the review complements with judgement).

Enforcement. Propagation gate `CM-COVENANT-114-125-PROPAGATION` enforces the literal anchor `11.4.125` across the consumer fleet (every CLAUDE.md / AGENTS.md / QWEN.md). Recommended gate `CM-CODE-REVIEW-GATE-BEFORE-BUILD` (the build orchestrator records a code-review-completed marker — with the review's findings + their remediation-or-no-blocking-findings determination — produced AFTER the batch's last fix and BEFORE the pre-build sweep + main build; a build that starts without a fresh review marker for the current batch FAILs).

Paired §1.1 meta-test mutations: strip the `11.4.125` literal → propagation gate FAILs; start a build with no fresh code-review marker for the batch → `CM-CODE-REVIEW-GATE-BEFORE-BUILD` FAILs. (Gate-code implementation lands as a separate follow-up work item; this anchor defines the contract.)

Classification: universal (§11.4.17) — "an independent multi-layer code-review-agent gate analyzing all work + data + codebase + git history for quality/safety/will-it-really-work + genuine test-validation, with findings blocking the build, runs BEFORE the pre-build sweep + main build" is a platform-neutral discipline reusable by ANY project that builds an artifact from source; the consuming project supplies its concrete review-marker mechanism + build-orchestrator integration per §11.4.35.

Canonical authority: this Constitution.md §11.4.125 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-code-review`, `--build-without-review`, `--no-review-gate`, `--review-optional`, `--trust-the-author` flag exists.

2026 06 08

§ . . — Code-review iterate-until-GO + rock-solid-evidence mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-08): "For any fixes/changes given back to us for re-work by the code-review process, once we fix/improve everything per the code-review's requests, we MUST RE-RUN code-review AGAIN until we get a GO from it with NO new issues reported or warnings of any kind! All results produced by this whole process MUST ALWAYS give us rock-solid PHYSICAL evidence that the fixed/improved codebase really works now as expected, with no false results and no bluff(s) of any kind."

When the §11.4.125 code-review returns ANY finding — BLOCKING, nit, or warning — and the author fixes/improves the batch per that review, the code review MUST BE RE-RUN, and MUST KEEP being re-run after each remediation round, until it returns a clean GO with ZERO new issues AND ZERO warnings of any kind. A single pass that "addressed the findings" is NOT sufficient: the corrected batch MUST pass a FRESH adversarial review (a re-review can surface NEW findings

introduced by the very fixes that closed the prior ones — the §11.4.1 fix-A-creates-B failure mode). The loop terminates ONLY on a clean GO (no new findings, no warnings); a residual warning is itself a finding that re-arms the loop.

Every round's verdict AND every fix's validation MUST carry rock-solid PHYSICAL captured evidence per §11.4.5 / §11.4.69 / §11.4.107 (captured audio / video / sysfs / dumpsys / sink-side / runtime-signature) proving the fixed/improved codebase REALLY works as expected on the target — never metadata-only / configuration-only / absence-of-error / grep-without-runtime. No false results, no bluff of any kind, at any round. A reported GO unbacked by captured physical evidence is itself a §11.4 PASS-bluff at the review-loop layer.

EXTENSION (2026-10-05): this clause's re-run obligation ("MUST KEEP being re-run after each remediation round, until it returns a clean GO") is bounded PER APPROACH by §11.4.276's declared round budget; at that ceiling the item is escalated per §11.4.276(E)(3) to a recorded operator decision, never closed, never continued round-by-round indefinitely under the same approach, and never silently relaxed to "good enough."

§11.4.134 REFINES / STRENGTHENS §11.4.125 (which requires the review to iterate "until no blocking findings remain"): it makes the loop EXPLICIT (re-run after every remediation round, not once), raises the termination bar to ZERO findings AND ZERO warnings (not merely zero-blocking), and BINDS rock-solid physical evidence to every round (not just the final one). Classification: universal (§11.4.17). Composes §11.4.125 / §11.4.1 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.69 / §11.4.107 / §11.4.50 / §11.4.108 / §11.4.123. Propagation gate `CM-COVENANT-114-134-PROPAGATION` (literal `11.4.134`) + recommended gate `CM-CODE-REVIEW-ITERATE-UNTIL-GO` + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: this Constitution.md §11.4.134 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-rereview`, `--single-review-pass`, `--warnings-ok`, `--evidence-optional` flag exists.

§ . . — Universal code-review mandate — every change reviewed, always, no exception (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-09):

> "ALL changes we do MUST pass through the code review step!!! ALWAYS!!!"

EVERY change made to ANY repository this Constitution governs — without exception — MUST pass through an INDEPENDENT code-review step BEFORE it is accepted, committed, or built. There is NO change class exempt: source code, fixes, tests, gates, meta-test mutations, documentation, doc-tooling, build/CI scripts, configuration, governance files (Constitution / CLAUDE / AGENTS / QWEN), conductor main-stream edits, sub-agent output, refactors, single-line edits — all of it. This is the ABSOLUTE form of §11.4.125 (which mandated a code-review agent gate after a batch of fixes, before the pre-build sweep + main build): §11.4.142 removes every implicit scoping that §11.4.125's "after all fixes/changes/ implementations are done" phrasing could be read to leave open — there is no "this is just a doc edit" exemption, no "this is just a one-liner" exemption, no "the author already self-reviewed (§11.4.92)" substitution, no "trivial change" carve-out. If a diff exists, it gets an independent review.

Independence is load-bearing. The review MUST be performed by a reviewer structurally separated from the author (a dedicated code-review agent, subagent-driven by default per §11.4.70 / §11.4.20, or a distinct human reviewer) — NOT the author re-reading their own work. §11.4.92's multi-pass self-evaluation is the AUTHOR-side discipline and remains required, but it does NOT satisfy §11.4.142: the whole point of an independent review seam is to catch the blind spots self-review cannot (the structurally-separated reviewer seam §11.4.70 names). A self-review labelled as the code-review step is a §11.4.142 violation of the same class as a §11.4 PASS-bluff at the review layer.

The review is itself anti-bluff. A rubber-stamp "looks good" with no examined findings is not a review (§11.4 / §11.4.1) — the review MUST genuinely analyse the change for correctness, safety (no host §12 / data §9 / target-hardware §11.4.133 regression), will-it-really-work (no solve-A-create-B per §11.4.1), end-user behaviour (§11.4 / §107), and test-genuineness (every covering test exercises the

work and catches its negation per §1.1). Its conclusions are captured evidence per §11.4.5 / §11.4.69. **The review iterates to a clean GO per §11.4.134** — any finding (BLOCKING / nit / warning) re-arms the loop; the change is accepted only after a re-run review returns ZERO new findings AND ZERO warnings, with rock-solid physical evidence per §11.4.134.

Composition, not duplication. §11.4.142 is the universal "always, every change, no exception" capstone over the code-review family: §11.4.125 supplies the multi-layer review CONTENT (the six analysis dimensions + the build-gate marker `CM-CODE-REVIEW-GATE-BEFORE-BUILD`); §11.4.134 supplies the iterate-until-GO + rock-solid-evidence LOOP; §11.4.92 supplies the author-side self-review that PRECEDES (never replaces) the independent review; §11.4.70 / §11.4.20 supply the subagent-driven default execution model. §11.4.142 binds them and strips the last exemption: the review step is not a batch-end nicety, it is a precondition on the ACCEPTANCE of any diff. Honest boundary (§11.4.6): a passing review proves the change was independently examined and found sound to the reviewer's evidence — it does NOT replace the §11.4.108 four-layer runtime-signature verification nor the §11.4.40 full-suite retest; it is one of MULTIPLE STRONG LAYERS, and the FIRST one every change crosses.

Forensic FACT (teaching citation, 2026-07-22 — review-before-push is not optional). A transport-hardening change was committed AND pushed to a project's `main` BEFORE any independent review (the author honestly flagged the omission); the post-hoc §11.4.209-substrate review then returned CONDITIONAL/NO-GO with ONE BLOCKING finding — the change had left a security-relevant protection default in the unsafe direction (an operator-keystroke-obscuring option shipped default-off with no disclosure) — plus two Important and four Minor findings, and the remediation consumed a full fix-forward cycle (RED-first test updates per §11.4.115, the launcher fix, three documentation surfaces, and a §11.4.134 re-review) that a pre-push review would have folded into the original change. This is the exact class this anchor forbids: the review found a real security-default defect that was ALREADY public on every mirror when it was found. The review comes BEFORE commit/push; a post-hoc review that catches a Blocking issue PROVES the mandate, it is not a substitute for it (remediation is fix-forward per §11.4.113 — never a history rewrite).

Classification: universal (§11.4.17) — a platform-neutral acceptance-gate discipline reusable by ANY project; the consuming project supplies its concrete reviewer-dispatch mechanism + the change-acceptance seam (commit wrapper / merge queue / PR gate) per §11.4.35. Composes §11.4.1 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.20 / §11.4.40 / §11.4.69 / §11.4.70 / §11.4.92 / §11.4.108 / §11.4.110 / §11.4.125 / §11.4.134 / §107 / §1.1. Propagation gate `CM-COVENANT-114-142-PROPAGATION` (literal `11.4.142` across the consumer fleet) + recommended gate `CM-EVERY-CHANGE-REVIEWED` (every accepted change carries a fresh independent-review-completed marker for its diff, produced by a reviewer distinct from the author, before acceptance/commit/build) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; accept a diff with no independent-review marker → `CM-EVERY-CHANGE-REVIEWED` FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.142. Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-review`, `--no-review`, `--trivial-change-exempt`, `--doc-edit-exempt`, `--self-review-suffices`, `--review-after-commit` flag exists.

11 4 145

2026 06 10

§ . . — Independent multi-angle impact-research per change (User mandate, - -)

Forensic anchor — verbatim operator intent (2026-06-10): for EVERY fix / change / new feature, independent agents MUST research potential issues related to that work AND connected/dependent features, from VARIOUS ANGLES, to be SURE no new issues are created, no existing working functionality is broken, and no dangerous code is introduced "like we had with the recents feature", incorporating a VARIETY OF TOOLS.

Forensic anchor (FACT, project-internal): the recents / 3-button-nav path shipped a latent AIDL-interface-contract mismatch that PASSED code review (the diff "looked correct") yet failed at RUNTIME ONLY — it ANR'd on a recents-reaping gesture — because no review angle interrogated the interface contract, the concurrency/lifecycle, or the connected feature it silently broke; the §11.4.108 SOURCE→ARTIFACT→RUNTIME→USER-VISIBLE gap, here caught only after the fact.

§11.4.145 shifts blast-radius discovery LEFT: for EVERY change, INDEPENDENT impact-research agents (subagent-driven §11.4.70/§11.4.20, structurally separate from the author — §11.4.92 self-eval PRECEDES, never satisfies — adversarial "refute-the-change" stance to defeat the documented LLM confirmation-bias failure mode where reviewers accept plausible-but-wrong arguments) research the change AND its connected/dependent features across a CLOSED SET OF EIGHT ANGLES — (1) **correctness/logic**, (2) **regression** (what existing working feature could break — via call-graph impact enumerating every direct + transitive caller and contract-dependent feature, each mapped to its test), (3) **latent/dangerous-code — the recents class** (runtime-only failure, interface/ABI/contract mismatch, concurrency/data-race, resource lifecycle), (4) **security** (taint/injection/secret/unsafe-API), (5) **performance** (latency/memory/thermal under load vs baseline), (6) **host/data/target-hardware safety** per §12/§9/§11.4.133, (7) **cross-feature interaction** (shared state/timing/hardware contention), (8) **business-logic conformance** (matches the spec AND the connected features' contracts, not merely compiles).

Each angle MUST name the TOOL(S) used + obtain the required DEPTH of codebase + business-logic detail (read the diff, the full changed unit, its declared contract, the spec section, and every connected feature — NEVER the diff lines alone) + emit a captured-evidence conclusion per §11.4.5/§11.4.69 (the tool's findings/impact/diff artefact path — "looks fine" is forbidden, §11.4.1). A genuinely-N/A angle is recorded `NOT_APPLICABLE: <reason>` per §11.4.6, never silently skipped. The output is a per-change impact-research REPORT (one section per angle: tool + evidence path + verdict) + a single GO/NO-GO that BLOCKS acceptance/commit/build on ANY unmitigated risk in ANY angle; the change is fixed/mitigated and the affected angles re-researched, iterating to a CLEAN GO (zero unmitigated risk + zero warning, rock-solid physical evidence) per §11.4.134 before the §11.4.125 review gate.

Honest boundary (§11.4.6): a GO proves cross-angle internal-consistency + bounded blast-radius — it does NOT replace §11.4.108 four-layer runtime-signature verification nor the §11.4.40 full-suite retest; it is one of MULTIPLE STRONG LAYERS, the FIRST research pass every change crosses.

Classification: universal (§11.4.17) — the consuming project supplies its concrete toolkit (static-analyzer, call-graph engine, interface-freeze mechanism, behavioral harness) + per-angle depth per §11.4.35. Composes/strengthens §11.4.92 (adds

independent tool-backed angle research) / §11.4.125 (its impact-research input) / §11.4.108 (shifts blast-radius discovery left) / §11.4.110 (angles 2/3/7 use the build-graph + interface-freeze) / §11.4.134 (iterate-to-GO) / §11.4.78 / §11.4.79 (call-graph impact engine) / §11.4.107 / §11.4.85 (behavioral angle) / §11.4.133 / §12 / §9 (angle 6) / §11.4.99 (cite latest sources) / §11.4.6 / §1.1. Propagation gate `CM-COVENANT-114-145-PROPAGATION` (literal `11.4.145` across the consumer fleet) + recommended gate `CM-IMPACT-RESEARCH-PER-CHANGE` (every accepted change carries a fresh independent impact-research report covering all eight angles with a clean GO, produced by agents distinct from the author, before acceptance/commit/build) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; accept a change with a report missing an angle or with an unmitigated NO-GO → `CM-IMPACT-RESEARCH-PER-CHANGE` FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.145. Non-compliance is a release blocker. No escape hatch — no `--skip-impact-research`, `--single-angle-suffices`, `--author-researches-own-change`, `--diff-skim-OK`, `--no-go-overridable`, `--trivial-change-no-research` flag.

2026 06 21

§ . . — Universal Independent Verification Agent Mandate (User mandate, - -)

Forensic anchor — direct user mandate (2026-06-21).

The mandate. Every code change OR recorded media artifact produced by any project governed by this Constitution MUST pass through an INDEPENDENT verification agent structurally separate from the author (§11.4.70/§11.4.20, dedicated subagent or distinct human, NEVER the author re-reading their own work — §11.4.92 self-evaluation PRECEDES and never satisfies). The verifier is one of MULTIPLE STRONG LAYERS — the FIRST verification layer every output crosses, complementing (never replacing) §11.4.108 runtime-signature verification, §11.4.125/§11.4.142 code review, and §11.4.40 full-suite retest.

(a) For CODE changes. The independent verification agent MUST: (i) run the §11.4.142 code-review discipline (adversarial analysis, blast radius, contract integrity, test genuineness), (ii) build and test the changed packages (invoke the

project's `pre_build_verification.sh` on the diff scope), (iii) run the paired §1.1 meta-test mutations for every gate the change touches (asserting each mutated gate still FAILs and its restoration still PASSes), (iv) verify the §11.4.108 runtime-signature registry entry for this change declares one machine-checkable observable. A code change that passes only one of (i)–(iv) is NOT verified — all four must hold.

(b) For MEDIA artifacts (video/audio/screenshots/asciicast/text output). The independent verification agent MUST: (i) run the §11.4.163 media validation pipeline on every recorded artifact, (ii) confirm the content is genuine — real LLM/service responses, not mocked/stub/simulated/placeholder (per §11.4.159(I)), (iii) confirm the artifact format is correct (MP4 H.264 for video, WAV/FLAC for audio, PNG for screenshots, cast+mp4 for asciicast per §11.4.159(B)), (iv) cross-check the evidence path against the artifact (the path exists, is non-empty, and the content matches the feature's expected output). A media artifact that passes format but shows simulated/placeholder content is a finding, not a PASS.

(c) For DOCS changes. The independent verification agent MUST: (i) validate that HTML + PDF + DOCX exports exist and are current per §11.4.65 (mtime parity), (ii) verify the §11.4.44 revision header is present and its integer has incremented from the previous commit, (iii) for docs that carry a §11.4.86 drift-proof fingerprint, verify the fingerprint matches the sorted roster/keyset. A doc change with a stale export or missing revision header is NOT verified.

(d) For CONFIG changes. The independent verification agent MUST: (i) validate YAML/JSON/TOML syntax (parse with the language's own parser — `yq`, `jq`, basic JSON parser, `python -m json.tool`, etc.), (ii) cross-reference against a schema where available (e.g. JSON Schema for `.settings.json`, YAML Schema for `helix-deps.yaml`), (iii) confirm no credential or secret pattern leaked per §11.4.10. A config change with a syntax error, schema violation, or leak is a finding, not a PASS.

(e) Structured findings. The verifier MUST produce a structured findings report — for each finding: item (which code/media/doc/config), verification class (a–d), tool/command used, evidence path (log file, build output, mutation result, validator verdict), PASS/FAIL. "Looks good" without evidence is not a finding — a rubber-stamp verifier is a §11.4 PASS-bluff at the verification layer.

(f) Iterate to GO. If any finding exists (PASS/FAIL indeterminate, BLOCKING, or even a warning), the verifier iterates per §11.4.134 — the author fixes the finding, the verifier RE-VERIFIES, until GO with ZERO findings and ZERO warnings. A single-pass verifier that reports findings but accepts GO because "the findings were minor" is a §11.4.165 violation.

(g) Anti-bluff — verifier self-validation. The independent verification agent is itself anti-bluff. The verifier's meta-test is a paired §1.1 mutation that introduces a deliberately broken code change (e.g. a gate with a stripped assertion) and asserts the verifier catches it. A verifier that passes a known-broken change is a bluff gate and must be investigated per §11.4.102 before any further use.

Honest boundary (§11.4.6). The independent verification agent confirms code/media/docs/config integrity at the source/artifact layer — it does NOT prove the feature works for the end user on the deployed target (that rests on §11.4.108 runtime-signature + §11.4.130 validate-fix-first + §11.4.40 full-suite retest) and does NOT replace the §11.4.40 full-suite retest or §11.4.118 discovery pass. A verifier that reports GO on an artifact whose downstream deployment is broken is not a verifier failure — that is caught by the later §11.4.108 layer. A verifier that reports GO without having actually run any tool (rubber-stamp) IS a verifier failure and a §11.4.165 violation.

Classification: universal (§11.4.17) — the consuming project supplies its concrete build+test command per diff scope, mutation-test runner, media validator invocation, doc export checker, config parser, and verifier-dispatch agent per §11.4.35. Composes §11.4.142/.163/.44/.65/.86/.108/.125/.134/.10/.40/.92/.70/.20/.102/.159(I)/.159(B)/§1.1. Propagation gate `CM-COVENANT-114-165-PROPAGATION` (literal `11.4.165`) + recommended gate `CM-INDEPENDENT-VERIFICATION-AGENT` (every output — code/media/docs/config — passes an independent verifier structurally separate from the author, produces structured findings with evidence paths, iterates to zero-finding GO; verifier is self-validated by a paired §1.1 mutation) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; accept an output without the independent verifier's GO → `CM-INDEPENDENT-VERIFICATION-AGENT` FAILs; verifier reports GO on a known-broken fixture → self-validation FAILs; gate-code = separate work item).

§ . . — Exhaustive all-scenario, all-angle code-review + verify-against-captured-runtime-evidence mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-07-14):

> "CRITICAL: Code review MUST TAKE into the accounts all scenarios, look at the problem or change from all angles, deepest possible analisys! No gaps, shortcomings or false results or bluff is allowed! Extend our root constitution Submodule and all relevant files properly so the review process is more precise and we do not have situations like the last one!"

Forensic case study (genericised, 2026-07-14 — "the last one"). A code-review (§11.4.125) returned a clean **GO with ZERO findings** on a boot-crash fix. It dismissed a defensive clamp branch as an INFORMATIONAL NIT — "effectively unreachable ... the input is always valid" — WITHOUT PROVING that assumption against the actual runtime input space. The fix shipped, was deployed to the target, and the target STILL crash-looped: the degenerate input the review had assumed could not happen DID happen at runtime (an empty / default-initialised profile opened with the exact zero / default value the review assumed impossible). The crash was a MULTIPLICATIVE size computation — a product `A × B` (e.g. `frame_size = channels × bytes_per_sample(format)`) — with TWO INDEPENDENT zero-triggers; the review verified only ONE factor (`A`, e.g. `channels`) and ASSUMED THE OTHER (`B`, e.g. `format`) away. The crash-loop reproduced on the deployed target under conditions PRESENT in the pre-fix crash evidence that the review never cross-checked the fix against — the evidence was available and not consulted.

Every code-review this Constitution governs (STRENGTHENING §11.4.125 / §11.4.134 / §11.4.142 — the review remains NECESSARY; §11.4.194 makes it EXHAUSTIVE-and-evidence-grounded so a clean GO can no longer be returned on an unproven assumption) MUST, before returning GO:

(1) Enumerate the FULL input / scenario space of BOTH the change AND the defect it addresses — every dimension, every code path, every interacting feature, every input domain the change touches — NOT merely the reported / happy-path

scenario. For a MULTI-FACTOR failure mode (e.g. a product of `N` terms, ANY of which being degenerate — zero, empty, null, default-initialised, out-of-range — triggers the failure), the review MUST enumerate AND INDEPENDENTLY VERIFY the fix covers EACH factor / dimension. Verifying ONE factor and ASSUMING the rest are safe is the EXACT forbidden gap this anchor closes (the forensic `channels`-verified / `format`-assumed pattern).

(2) Prove every assumption. "This can't happen" / "unreachable" / "always valid" / "the caller never sends X" / "the input is always in range" MUST be PROVEN from the code + spec + captured evidence — NEVER accepted on faith, NEVER inferred from "it usually is" (§11.4.6 no-guessing). An unproven assumption that later proves false is a §11.4 REVIEW-LAYER bluff of the SAME severity class as a PASS-bluff. A review NIT that DISMISSES a path / branch as unreachable / dead / defensive-only MUST carry the PROOF of unreachability (the callers enumerated, the input domain bounded, the degenerate value shown genuinely impossible) — a bare "effectively unreachable" NIT with no proof is ITSELF a finding, not a dismissal.

(3) Verify the fix against ANY captured runtime evidence. When captured runtime evidence of the failure EXISTS — a crash log / tombstone / stack trace / core dump / sink-probe report / the §11.4.108 runtime-signature / a reproducer — the review MUST confirm the fix ACTUALLY PREVENTS the CAPTURED failure (walk the fix against the EXACT captured conditions, not a hypothetical). A review that GO's a fix which then FAILS at runtime on a scenario PRESENT in the evidence (or a scenario the review CHOSE NOT TO CONSIDER) is a bluff — the evidence was available and not cross-checked.

(4) All angles. The review MUST cover ALL of: correctness against the stated goal; the FULL input domain (every factor of a multi-factor computation, every boundary, every degenerate value); every downstream / interacting feature — ESPECIALLY must-not-break constraints (per §11.4.133 target / hardware safety, and any feature the change shares state / timing / a resource with — e.g. audio / multichannel / passthrough integrity); the TESTS' OWN coverage of every dimension (a test that checks only ONE factor of a multi-factor bug is ITSELF a coverage gap the review MUST catch and require closed); and the §1.1 paired mutation covering EACH dimension (a mutation that flips only one factor leaves the others un-guarded).

(5) Honest boundary (§11.4.6). "Exhaustive" means the ENUMERATED + EVIDENCE-GROUNDED scenario space — NOT a claim of literal omniscience. A dimension that genuinely cannot be analysed (external state the review cannot

inspect, a runtime path with no reproducer) MUST be stated as an EXPLICIT gap (`UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:` with a tracked follow-up) — NEVER silently assumed safe. A silently-assumed-safe un-analysed dimension is the §11.4.194 violation.

(6) The quality-harvest review classes (research-derived extension, 2026-07-23 — operator mandate: every check, diagnostic method, and preventive measure from the reopen/first-touch root-cause forensics is checked through EVERY code-review process). In addition to (1)–(5), every review MUST explicitly CHECK, per reviewed change: **(a) closure-evidence class vs defect layer (§11.4.226)** — no terminal or ready-class status in the change's scope moves on evidence whose class ranks below the defect layer's floor; a user-visible property "verified" by file-existence checks plus greps, an echo-assertion, or a state-only repair closing a defect as fixed is a FINDING; **(b) needled measurements** — every load-bearing measurement the change or its tests performs satisfies §11.4.201(6)–(12) (a reported absence cites its class-matched control needle), and NO count was converted into a finding without reading the underlying lines — a COUNT IS A LEAD, THE LINES ARE THE FINDING (forensic FACT: twice in one measured cycle a count was read as state — a "mixed half-written doc" that was actually coherent with a labelled historical citation, and an inflated FAIL total that included the summary line counting itself); **(c) governance-and-gate integrity (§11.4.227)** — a change that NAMES a gate lands its implementation or a registered deferral in the same change, and a governance edit preserves anchor-block integrity (exactly-once per mirror, lockstep-identical, no anchor-number collision); **(d) reviewer-authored adversarial mutations** — the author's self-authored mutation set is NECESSARY but NOT SUFFICIENT validation of a guard/test: forensic FACT (from the consuming project's dated forensic corpus cited in the 2026-07-22 root-cause analysis): an author's six self-authored mutations were ALL caught while an independent reviewer's FIRST THREE different mutations ALL sailed through, each restoring a user-visible defect. The reviewer MUST attempt at least one mutation the author did not write (or record honest infeasibility per §11.4.3); a guard whose reviewer-authored mutation survives is unvalidated instrumentation per §11.4.115(F) and a FINDING.

Classification: universal (§11.4.17) — a platform-neutral review-rigor discipline reusable by ANY project; the consuming project supplies its concrete input registries, must-not-break constraint set, and captured-runtime-evidence sources per §11.4.35. Composes §11.4.125 (the code-review-agent gate this strengthens) /

§11.4.134 (iterate-until-GO — a §11.4.194 gap is a finding that re-arms the loop) / §11.4.142 (every-change-reviewed — §11.4.194 raises the review's PRECISION bar) / §11.4.6 (no-guessing — the operative discipline behind clauses 2 + 5) / §11.4.108 (runtime-signature — the captured evidence clause 3 cross-checks against) / §11.4.130 (validate-the-fix-first-after-redeploy — the runtime confirmation that would have caught "the last one") / §11.4.107(10) (self-validated analyzer — the multi-factor coverage discipline applied to the review's OWN test-coverage check). Propagation gate `CM-COVENANT-114-194-PROPAGATION` (literal `11.4.194` across the consumer fleet) + recommended gate `CM-EXHAUSTIVE-REVIEW-ALL-SCENARIOS` (a review GO must record the enumerated scenario / factor space + the proof of every dismissed-as-unreachable branch + the cross-check against captured runtime evidence where it exists) + paired §1.1 meta-test mutation (strip the enumeration / the assumption-proof / the captured-evidence cross-check from a review record → the gate FAILs; strip the literal → the propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.194.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--partial-scenario-review`, `--assume-unreachable-OK`, `--skip-captured-evidence-crosscheck`, `--single-factor-review-suffices`, `--dismiss-branch-without-proof` flag.

§. . — Code-review MUST run on the Opus model at xhigh effort as PRIMARY substrate, with a genuine-unavailability Sonnet fallback — no Fable, no escalation tier (User mandate, - - , supersedes - - and - - ; amended - - to add the Sonnet-fallback clause)

Verbatim operator mandate (2026-10-04, `CRITICAL`), AMENDING clause (A) below — adds a genuine-unavailability fallback, does not change the primary substrate: "If we do not have access to Opus model for some reason but only to Sonnet, we MUST USE what we have! Reviews MUST BE done. Ideally with Opus model, however Sonet MUST BE used if Opus for any reason cannot be used! Add

this / extend into main constiotution Submodule and apply the rule asap so nothing is blocked with review which is not performed yet! We MUST BE maximally efficient! Work MUST BE performed with all means we have at particular moment!"

Verbatim operator mandate (2026-09-26, `CRITICAL`), superseding both mandates below: "Main model to use is Sonet, reviews and other heavy stuff have to be done by Opus instead of Fable!!! Fix this permanently on all levels!"

Prior mandate (2026-09-15), WITHDRAWN as current, kept for lineage: *"the mandatory independent code-review MUST by default be performed with the Opus model at xhigh effort; for a review the operator or the reviewer itself classifies as extremely big or complex, the Fable model at xhigh effort MUST be used instead."* Its Opus-xhigh default survives; its Fable escalation clause (former clause (A)) is REMOVED by the 2026-09-26 mandate — there is no review tier above Opus-xhigh.

Prior mandate (2026-07-15), WITHDRAWN as current, kept for lineage: *"the mandatory independent code-review MUST ALWAYS be performed with the Fable model at xhigh effort; if Fable is unavailable, use Opus at xhigh effort."* Inverted by the 2026-09-15 mandate and fully superseded by the 2026-09-26 mandate.

Every mandatory independent code-review the constitution requires — the §11.4.125 code-review-agent gate before pre-build + main build, the §11.4.142 universal every-change-review (no exception), and the §11.4.194 exhaustive all-scenario/all-angle review + verify-against-captured-runtime-evidence, iterated to a zero-finding GO per §11.4.134 — MUST be dispatched on the **Opus model at `xhigh` effort** as PRIMARY substrate, whatever the size or complexity of the reviewed change. **(A) PRIMARY substrate Opus-xhigh + genuine-unavailability SONNET FALLBACK (amended 2026-10-04) — no escalation tier, no Fable, no other fallback model.** Opus at `xhigh` is the PRIMARY and preferred review substrate. Fable is NOT permitted in any role (default, escalation, or fallback), ever; no model other than Opus or the sanctioned Sonnet fallback below, and no lower tier for Opus itself, substitutes for it. When Opus at `xhigh` is genuinely unavailable (not configured / rate-limited / unreachable, established as a captured FACT per §11.4.6 — never a guessed unavailability), the review MUST be dispatched on **Sonnet** (at the highest effort the dispatch path can set) rather than blocked — the sole sanctioned fallback, per the verbatim 2026-10-04 operator mandate above: reviews MUST be done with whatever capable means are actually available, Opus ideally, Sonnet when Opus genuinely cannot be used. The Opus-unavailability

FACT and the Sonnet-fallback substitution MUST both be recorded honestly in the review's captured evidence (§11.4.5/§11.4.69) — a Sonnet-fallback review is NEVER silently presented as if it ran on Opus. A review is NEVER left undone / blocked / deferred solely because Opus is unavailable while Sonnet is reachable — maximal efficiency (§11.4.183) forbids leaving genuinely performable review work undone. ONLY when BOTH Opus and Sonnet are genuinely unavailable (each established as its own captured FACT per §11.4.6, never guessed) does the review become BLOCKED and deferred under the original (A) terms: the unavailability fact is recorded, the owed review is tracked (§11.4.197; §11.4.21 once the self-resolution audit is exhausted), and the reviewed change does NOT pass the review gate unreviewed. **(B) Independence preserved (§11.4.70/§11.4.20).** The reviewing model (Opus or the sanctioned Sonnet fallback) runs as the structurally-separated reviewer (never the author self-reviewing); §11.4.209 sets the MODEL + EFFORT of that review, it does not weaken the independence, the iterate-until-GO loop (§11.4.134), or the rock-solid captured-evidence requirement (§11.4.5/§11.4.69/§11.4.107). **(C) Honest boundary (§11.4.6).** A review claimed "done" that actually ran on Fable, Haiku, or any model other than Opus or the sanctioned Sonnet fallback, or at an effort below `xhigh`, or on Sonnet with no recorded Opus-unavailability fact, is a §11.4 bluff at the review-substrate layer — the review's captured evidence MUST record which model + effort actually performed it. Where the dispatch path cannot set or report the effort (the §11.4.231(F.2) harness gap), the evidence records that gap honestly and the review is never claimed to have run at `xhigh` on an unverified basis. **(D) Values already burned in a public repository (§11.4.10).** Any plaintext credential value that has appeared in documentation, a spec, or commit history — including as a worked example — MUST be treated as compromised from the moment of commit; a document illustrating this anchor or any credential-adjacent mandate MUST NOT embed a real-looking username/password pair, even as an example, without the accompanying statement that the pair is compromised-by-disclosure and MUST NOT be relied on as a real secret. STRENGTHENS §11.4.125/§11.4.142/§11.4.194/§11.4.134 — it does NOT replace any of them; it pins the review's execution substrate, now a PRIMARY-plus-sanctioned-fallback pin rather than a single-model pin.

Classification: universal (§11.4.17). Composes §11.4.125 / §11.4.134 / §11.4.142 / §11.4.194 / §11.4.6 / §11.4.10 / §11.4.20 / §11.4.21 / §11.4.70 / §11.4.183 (maximal multi-agent utilization — the reason a genuine Opus gap defers to Sonnet rather

than blocking, never reduces to idle waiting) / §11.4.196 (alias/model selection) / §11.4.197 / §11.4.231 (nano-precision model-tier selection — §11.4.209 remains the hard pin for review/merge-conflict scope that §11.4.231(E) explicitly does not reach; the amended clause (A) Sonnet fallback draws on the SAME `sonnet` rung §11.4.231(A) already names, never a new tier). Propagation gate `CM-COVENANT-114-209-PROPAGATION` (literal `11.4.209` across the consumer fleet) + recommended gate `CM-CODE-REVIEW-OPUS-XHIGH` (renamed by the 2026-09-26 amendment; the retired predecessor token is cited in `scripts/gates/gate_ledger_removals.tsv`; amended 2026-10-04) (the mandatory code-review's captured evidence records model=Opus + effort=xhigh (or, on a dispatch path that cannot set effort, that §11.4.231(F.2) harness gap honestly recorded instead of a claimed xhigh), OR model=Sonnet carrying a captured Opus-unavailability FACT under the SAME effort-or-honest-gap rule; a review recorded on Fable/Haiku/any other model, recorded as xhigh when the dispatch path could not actually set it, or recorded on Sonnet with NO captured Opus-unavailability fact, FAILs; a review recorded BLOCKED with a captured fact that BOTH Opus AND Sonnet were unavailable is honest and MUST NOT fire the gate — the §11.4.201(1) false-positive guard) + paired §1.1 meta-test mutation (record a code-review run on a non-Opus/non-Sonnet model, or claiming xhigh when the dispatch path could not actually set it, or on Sonnet with no recorded Opus-unavailability fact → the gate FAILs; strip the literal → the propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.209.

Non-compliance is a release blocker. No escape hatch beyond the sanctioned (A) Sonnet fallback (which itself requires a captured Opus-unavailability FACT, never a guess) — no `--review-any-model`, `--review-lower-effort-OK`, `--skip-opus`, `--non-xhigh-review`, `--escalate-to-fable-or-haiku`, `--sonnet-fallback-without-recorded-fact`, `--leave-review-blocked-while-sonnet-reachable` flag.

§ . . 2026 08 15 — Producer ≠ Verifier scope separation: the actor that produces a change/artifact/verdict cannot be the one that gates it — least privilege over cleverness (research-derived, - -)

Compact summary: every project MUST enforce STRUCTURAL SEPARATION between the actor that PRODUCES a change/artifact/verdict and the actor that GATES that same change/artifact/verdict — the producing actor cannot write, weaken, delete, or edit the acceptance tests, the oracle, the gate configuration, the verdict store, the paired §1.1 mutation, or any other check whose green result is a precondition for the producer's own work being certified done. Enforcement is by CAPABILITY (the producer STRUCTURALLY cannot reach the gate) rather than by INSTRUCTION (the producer is TOLD not to touch the gate), and the enforcement mechanism is LEAST PRIVILEGE OVER CLEVERNESS (extraction file line 1488 — **MODULE ATTRIBUTION CORRECTED 2026-08-20: this citation originally read "module 27". The quoted passage has ZERO occurrences in `module_27.txt` and resolves in the tracked corpus to module 28 ("Anti-Bluff Verification & Release Validation"), verified locator `module_28:393`. Instrument control-needled per §11.4.201(7)(b) before the absence was reported.** — module 28, verified locator `module_28:393` : "the durable fix is least privilege over cleverness — the same logic as prompt-injection defense ... the load-bearing move is that the agent chasing the green gate cannot reach the gate, the oracle, or the evidence store"). The failure mode this anchor forbids: a producer that CAN edit the tests/oracle/gate config/verdict store DOES edit them under long-horizon optimization pressure — hardcoding expected values, weakening assertions, mocking the unit under test, deleting the failing test, or writing `verified: true` on itself (extraction lines 5695, 5711-5717, 5722-5730, 1343, 1396, 1482-1488). §11.4.240 makes the separation a STRUCTURAL PROPERTY of the codebase and its automation, not a social one.

(A) THE FOUR SCOPES + THE SEPARATION MATRIX. Every change/artifact/verdict flows through four SCOPES that MUST be distinct actors under §11.4.240: (i) PRODUCER — the actor that writes the source diff, the config change, the migration, the artifact-generation code; (ii) ORACLE — the actor / mechanism that decides "correct output" for a given input (test author, specification writer, characterization-baseline capturer, reference-implementation author per §11.4.245 oracle-independence); (iii) GATE — the actor / mechanism that decides whether

the change PASSES the check (CI configuration, gate scripts, custody-chain enforcer per §11.4.146(D3), release-verdict-coverage machinery per §11.4.135); (iv) VERIFIER — the actor / mechanism that RE-RUNS the gate from a clean checkout and attaches receipts (independent verifier per §11.4.165, §11.4.209 Opus-xhigh substrate for code review, §11.4.194 exhaustive review). The mandate: the PRODUCER of a change cannot ALSO be the ORACLE, GATE, or VERIFIER for that same change. Extraction line 5695 (module 34 core concept 4 — module attribution VERIFIED CORRECT 2026-08-20, verified locator `module_34:56`; offset un-resolvable per the §11.4.239 CITATION NOTE): "the agent that wrote the code cannot be the authority that certifies it — self-preference bias and shared context make self-review worthless. Verification comes from an independent instance (fresh context, read-only, re-running from a clean checkout) and from an oracle the producer did not author and cannot edit ... Remove any one property and it degrades to self-certification." A four-scope collapse (one actor plays all four roles) is the maximum-severity form; any two-scope collapse involving the producer is a §11.4.240 finding of §11.4/§11.4.1 severity at the certification layer.

(B) LEAST PRIVILEGE OVER CLEVERNESS — CAPABILITY, NOT INSTRUCTION. Separation MUST be enforced by structural CAPABILITY LIMITS, not by prompts / instructions / policy documents / social contracts asking the producer "please do not touch the tests." Extraction lines 1482-1488 [MODULE CORRECTED 2026-08-20 — the (E) Extraction-sources line attributes these offsets to module 27; the quoted passage resolves to module 28, verified locator `module_28:383`] name the load-bearing mechanism: "The producer has no write access to the test files, the oracle, the CI/gate config, or the verdict store — enforced by scope, not by asking. If it can't touch the gate, it can't game it." Concrete enforcement channels the consuming project MUST supply per §11.4.35: (1) FILESYSTEM SCOPE — the producer's write scope excludes `tests/`, `oracle/`, `.github/workflows/`, `scripts/gates/`, `verdicts/` (or their consumer-declared equivalents); (2) BRANCH / REVIEW PROTECTION — protected paths (see §11.4.248 PROTECTED-SPEC + CODEOWNERS from the sibling 2026-08-15 operator decision) require an actor OTHER than the change author to approve modifications; (3) VERDICT-STORE ISOLATION — the store that records RED/GREEN verdicts per §11.4.115(F) is written only by the harness that ran the test, never by the producer's own code path; (4) EXECUTION ISOLATION — the verifier runs in a CLEAN CHECKOUT with FRESH CONTEXT (extraction lines 3168, 3117, 2381 [MODULE CORRECTED 2026-08-20 — the (E) Extraction-

sources line attributes these offsets to module 34; the quoted passage has ZERO occurrences in `module_34.txt` and resolves to module 29 / module 30, verified locators `module_29:655`, `module_30:591`, `module_30:655`]: "an independent verifier re-running in a clean checkout") — the verifier does NOT inherit the producer's dirty worktree, cached artifacts, or in-flight session state. "The agent will not do that" is NEVER acceptable enforcement — extraction line 5715 [module 34 attribution VERIFIED CORRECT 2026-08-20, verified locator `module_34:87`] catalogues the failure ("We told it in the prompt not to hardcode / not to skip tests" as the shallow / red-flag answer). Exhortation is not capability; §11.4.240 refuses exhortation as the enforcement layer.

(C) THE THREE FORBIDDEN PRODUCER↔SCOPE COLLAPSES. The following collapses are §11.4.240 violations regardless of context, scale, or claimed rigor: (1) PRODUCER=ORACLE — the same actor writes the source diff AND the test that asserts its correctness, with no independent oracle authoring the assertion; the resulting test asserts whatever the code happens to do ("tautological testing trap," extraction line 5764 [module 34 attribution VERIFIED CORRECT 2026-08-20, verified locator `module_34:155`], referenced) rather than what the specification requires. Countermeasure: the oracle is authored FROM the specification / from a characterization baseline captured BEFORE the change (§11.4.243) / from a reference implementation (§11.4.245 derived oracle), not co-authored with the diff. (2) PRODUCER=GATE — the same actor writes the source diff AND has write access to the gate configuration / CI pipeline / the mutation-testing thresholds / the paired §1.1 mutations; the producer can silence failing gates or weaken thresholds. Countermeasure: the gate configuration lives in a §11.4.248-protected path, and modifications require a non-producer approver. (3) PRODUCER=VERIFIER (self-certification) — the same actor produces the change AND writes the "verified: true" / "PASS" verdict for the change; the verdict is a producer-set flag with no independent re-run. Countermeasure: verdicts are MACHINE-WRITTEN by the harness that ran the test per §11.4.115(F), keyed to the target's artifact fingerprint read at run time, and the release-verdict-coverage seam per §11.4.135 REFUSES a release tag while any producer-asserted (rather than harness-written) verdict is in the release scope. Extraction lines 1212, 1343, 1396, 1631 [MODULE CORRECTED 2026-08-20 — attributed to module 27 in the (E) Extraction-sources line; the quoted passage resolves to module 28, verified locator `module_28:278`]: "Tamper-evident and outside the producer's reach — the producer of the change cannot edit the verdict or the oracle to make itself green."

(D) COMPOSITION WITH §11.4.115(F) / §11.4.125 / §11.4.135 / §11.4.142 / §11.4.146(D3) / §11.4.165 / §11.4.194 / §11.4.209. §11.4.115(F) machine-written verdict pairs are the (C)(3) countermeasure at the verdict layer — a producer-typed status label is not a verdict. §11.4.125's code-review-agent gate before pre-build + main build IS the (A) VERIFIER role at the review layer, and §11.4.142's universal every-change-review mandate + §11.4.209's Opus-xhigh-primary-plus-genuine-unavailability-Sonnet-fallback review-substrate pin (amended 2026-10-04; Fable and Haiku remain forbidden in every role) provide the independence + substrate requirements. §11.4.135's standing regression-guard suite is the GATE layer at the release seam. §11.4.146(D3) status custody is the STATUS-WRITE-SEAM enforcement — the store REFUSES a producer-asserted terminal-status write without the full custody chain. §11.4.165 (universal independent verification agent) is the VERIFIER role generalised across every output. §11.4.194 (exhaustive all-scenario review) is the depth requirement applied to the verifier's pass. §11.4.240 does NOT invent a new mechanism at any of these layers — it ARTICULATES the property (structural producer-vs-verifier separation) that the existing mechanisms collectively enforce, and it forbids the collapses (§11.4.240(C)) that would render those mechanisms decoration.

(E) HONEST BOUNDARY (§11.4.6). Structural separation forbids the CLASS of self-certification failure modes; it does NOT prove the verifier is correct (a mis-calibrated verifier stays §11.4.107(10) golden-good/golden-bad territory; extraction line 1512 [MODULE CORRECTED 2026-08-20 — implicitly module 27 via the (E) Extraction-sources line; the quoted passage resolves to module 28, verified locator `module_28:401`]: "a flaky or false-positive gate destroys the whole evidence chain above it"), does NOT prove the oracle is correct (§11.4.245 mandates oracle-strategy naming + independence), and does NOT guarantee the producer never tries to game the gate (it forbids the producer from succeeding at gaming the gate through direct write access; other gaming vectors — hardcoding to visible inputs, over-fitting the seen cases — are countered by §11.4.245 held-out oracles + property/metamorphic/differential coverage). Composes §11.4.6 (no guessing — the producer's claim is a prediction, only the verifier's captured re-run is evidence) / §11.4.20 (subagent-driven — the verifier IS the natural subagent dispatch) / §11.4.70 (subagent-driven default) / §11.4.92 (multi-pass evaluation — self-review PRECEDES independent review, never satisfies it) / §11.4.107(10) (self-validated analyzers — the oracle carries its own golden fixtures) / §11.4.115(F) (machine-written verdicts) / §11.4.125 (code-review-agent gate) / §11.4.134 (iterate-to-GO —

the verifier's clean-GO terminates the loop) / §11.4.135 (standing regression-guard suite) / §11.4.142 (universal every-change-review) / §11.4.146(D3) (status custody chain refuses producer-asserted terminals) / §11.4.165 (universal independent verifier agent) / §11.4.194 (exhaustive review) / §11.4.201 (a gate that false-refuses a clean producer is a FAIL-bluff, so verifier honesty binds both directions) / §11.4.205 (seam-not-prose enforcement) / §11.4.209 (Opus-xhigh substrate pin for the review verifier) / §11.4.245 (oracle-strategy identification, independent by the same principle) / §11.4.248 (PROTECTED-SPEC + CODEOWNERS enforcement — the operator-decided mechanism for (B)) / §11.4.249 (four-role architecture + flight-recorder mechanism — §11.4.240 is the SEPARATION PRINCIPLE + capability enforcement, §11.4.249 is the concrete FOUR-ROLE {producer, oracle, gate, verifier} taxonomy + recorder that instantiates it; the two anchors mutually compose per §11.4.209 reviewer MINOR-1). Extraction sources [offsets un-resolvable per the §11.4.239 CITATION NOTE; module attributions AUDITED **and CORRECTED** 2026-08-20]: the offsets formerly attributed to "module 27" (1212, 1343, 1396, 1482-1488, 1498, 1613, 1631-1634) are a WRONG-MODULE attribution — every phrase this anchor quotes from them resolves to **module 28** ("Anti-Bluff Verification & Release Validation"), verified locators `module_28:278` (tamper-evident / the producer cannot edit the verdict or the oracle to make itself green), `module_28:383` (no write access to the test files; enforced by scope, not by asking; if it can't touch the gate, it can't game it), `module_28:393` (least privilege over cleverness; the same logic as prompt-injection defense), `module_28:401` (a flaky or false-positive gate destroys the whole evidence chain above it); **module 34** ("Quality Under AI-Assisted / Agentic Development") lines 5695, 5711-5717, 5722-5730, 3180, 3201, 2313 are module-CORRECT for every phrase quoted from them — verified locators `module_34:56` (the agent that wrote the code cannot be the authority that certifies it), `module_34:87` (we told it in the prompt not to hardcode), `module_34:155` (tautological testing trap); the offsets **3117-3168 and 2381**, listed under module 34, are a WRONG-MODULE attribution — the phrase quoted from them ("an independent verifier re-running in a clean checkout") has ZERO occurrences in `module_34.txt` and resolves to **module 29 / module 30**, verified locators `module_29:655`, `module_30:591`, `module_30:655`. **UNKNOWN (§11.4.6, stated rather than guessed):** offsets 1498, 1613, 1631-1634, 5711-5717, 5722-5730, 3180, 3201 and 2313 carry no

quoted text in this anchor, so no phrase-level locator could be measured for them individually — their module attribution is UNVERIFIED: neither asserted correct nor asserted wrong.

(F) Independence-tier vocabulary + Tier-C capability-separation triggers, degrading honestly on a single-uid host (extension, spec-derived, speckit 002-anti-slop-enforcement, 2026-08-26). Clauses (A)–(E) state the SEPARATION PRINCIPLE and its capability-not-instruction enforcement in the abstract; this clause supplies the CLOSED VOCABULARY for recording HOW MUCH separation was actually achieved, and the two triggers that raise the required strength — because a project claiming "capability separation" where only "instance separation" is genuinely achievable is itself a §11.4.240 violation of the theatre this anchor exists to prevent. **(1) The closed tier set.** Per spec.md:124 (FR-035): every accepted verdict's recorded basis of independence MUST name which tier applied, drawn from the CLOSED SET `{instance, model, capability}` — `instance` (a separate agent/process instance verifies, sharing no in-flight state with the producer), `model` (a DIFFERENT model verifies — permitted, never required per (2) below), `capability` (a genuine host-level access-control boundary separates the producer's write scope from the verifier's — the strongest tier). **(2) Model diversity is permitted, never required — the false-positive guard.** Per spec.md:121 (FR-032): a verdict from an independent INSTANCE running the SAME model as the producer MUST be ACCEPTED on that basis alone (Tier B, permitted-never-required) — a check that refuses same-model independent verdicts has turned a legitimate separation into an unnecessary same-model prohibition, which is itself the exact class of false-positive refusal §11.4.201 forbids. This clause does NOT weaken the constitution's OTHER, distinct, narrower model-substrate pins — §11.4.256's requirement that a translation review run on a DIFFERENT model/provider than the translator, and §11.4.209's requirement that every mandatory independent code-review run on the Opus model at `xhigh` effort as PRIMARY substrate, with a genuine-unavailability Sonnet fallback (amended 2026-10-04; Fable and Haiku remain forbidden in every role); both remain in force unmodified for their own narrower domains (translation review; code review). FR-032's permitted-not-required framing governs only the GENERAL producer≠verifier case this anchor addresses, never those two domain-specific, stricter pins. **(3) Tier-C trigger 1 — the producer holds write access to the evidence/verdict store.** Per spec.md:122 (FR-033): at a seam where the producer holds write access to the evidence or verdict store, a verdict MUST be written

through the MOST RESTRICTIVE separation genuinely available on the host, and the ACHIEVED separation (not the aspired-to one) MUST be reported. **Measured, honestly-degrading limit (spec.md:122):** *"on a single-uid host, capability separation cannot be established by any mechanism an agent can set up — ownership defeats access-control lists, the append-only attribute requires privilege the agent lacks, and any key the producer can use it can also read. Claiming* `capability` *there would be the theatre this feature exists to prevent."* A single-uid host therefore reports `instance` honestly, NEVER `capability`, until the operator takes the one action that establishes a real host boundary. **(4) Tier-C trigger 2 — the three named high-blast-radius seams.** Per spec.md:123 (FR-034): at the `release-tag`, `qa-deploy`, or `manual-qa-handoff` seam (or the consuming project's declared equivalent seams per §11.4.35), a producer-authored verdict is REFUSED regardless of the write-access analysis, and the achieved separation is reported rather than assumed. **Same honest-degradation limit applies (spec.md:123):** *"Capability-strength at these three seams requires one operator action (a host boundary agents are correctly forbidden from creating); until it is taken they operate at* `instance` *tier and MUST say so."* **(5) No demand for capability outside the two triggers.** Per spec.md:125 (FR-036): at a seam matching NEITHER Tier-C trigger, a separate agent instance MUST SUFFICE and capability separation MUST NOT be demanded — instance-tier verdicts are accepted at ordinary seams without a capability check, closing the negative half of this clause (a project over-applying Tier-C strength to every seam is itself a §11.4.240 violation of the same "false-positive refusal" class). **Honest boundary (§11.4.6).** The `independence_tier` field records WHAT WAS ACHIEVED, never what was ASPIRED to — a project honestly reporting `instance` at a Tier-C-triggered seam because no host boundary yet exists is COMPLIANT with this clause (the honest-degradation path is the compliant path); a project reporting `capability` without the mechanically-verified boundary that makes it true is the violation. Composes §11.4.6 (honest reporting over aspiration) / §11.4.201 (a false-positive `capability` claim is the same class of bluff a false-positive guard refusal is) / §11.4.256 (its own, distinct, domain-specific different-model requirement stays unweakened) / §11.4.209 (its own, distinct, domain-specific Opus-`xhigh` code-review pin stays unweakened alongside §11.4.256's) / §12 (host-boundary establishment is a §12 host-safety-adjacent operator action, never an autonomous agent action). Recommended mechanism gate `CM-INDEPENDENCE-TIER-HONESTLY-RECORDED` (every accepted verdict's

`independence_tier` field is present, within the closed {instance, model, capability} set, and — at a Tier-C-triggered seam without a mechanically-verified host boundary — is NEVER `capability` ; a same-model independent verdict at an ordinary seam is accepted without a capability check) + paired §1.1 mutation (record `capability` at a seam where no verified host boundary exists → the gate MUST FAIL; refuse a same-model independent verdict at an ordinary seam → the gate MUST FAIL [the false-positive-guard case]; accept a producer-authored verdict at `release-tag` / `qa-deploy` / `manual-qa-handoff` → the gate MUST FAIL; golden-FALSE per §11.4.201(1): an `instance` -tier verdict honestly reported at a Tier-C-triggered seam with no host boundary yet established → the gate MUST NOT fire). Gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227).

Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-240-PROPAGATION` (literal `11.4.240`) + recommended gates `CM-PRODUCER-NOT-VERIFIER-SCOPE-SPLIT` (a change's four scopes per (A) — producer, oracle, gate, verifier — resolve to distinct actors; a producer-authored verdict, a producer-modifiable gate config, a producer-modifiable oracle, or a producer-modifiable test file in the same change violates the split) + `CM-LEAST-PRIVILEGE-CAPABILITY-NOT-INSTRUCTION` (the enforcement channel is a filesystem-scope / branch-protection / verdict-store-isolation / execution-isolation mechanism per (B), NOT an instruction / prompt / policy-document / social-contract; a project whose only enforcement is a written policy that "producers do not touch tests" fails the capability check) + paired §1.1 mutations (make the producer branch a modification to a `tests/*` file within the same change without a non-producer approver → the CODEOWNERS + capability check FAILs; author a status write of "verified: true" from the same actor that produced the diff, with no harness-written §11.4.115(F) verdict pair → the §11.4.146(D3) custody chain FAILs; grant the producer write access to the gate configuration + expect the review to catch the collapse → the capability-not-instruction gate FAILs on the enforcement-channel check; strip the literal → propagation FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227).

Canonical authority: constitution submodule `Constitution.md` §11.4.240. Non-compliance is a release blocker. No escape hatch — no `--producer-may-write-tests` , `--self-certification-OK` , `--exhortation-suffices-as-`

`enforcement`, `--producer-authored-verdict-accepted`, `--capability-limit-optional`, `--policy-document-is-enforcement`, `--single-actor-for-all-scopes` flag.

---

§ . . — Illegal-state-unrepresentability preference: types before API-shape before lint before property-test before runtime-assertion (research-derived, - -)

Full anchor text per Phase 3 landing brief. See `curriculum_anchors_241_254_full_drafts.md` §11.4.241 canonical block (Part A). Anchor establishes the closed ordered ladder (1) TYPE SYSTEM → (2) API SHAPE → (3) LINT → (4) PROPERTY-BASED TEST → (5) RUNTIME ASSERTION for every load-bearing invariant; a defect preventable at rung N caught only at rung > N is a finding. Composes §11.4.108 (pushes enforcement LEFT of Layer 1) / §11.4.115 / §11.4.142 / §11.4.194 / §11.4.201 / §11.4.224 / §1.1 / §11.4.6 / §11.4.35. Includes rung-by-language capability baseline (strongly-typed: rungs 1-5; dynamically-typed with gradual: rung 1 conditional; untyped scripting: rungs 3-5 unconditional); genuinely-limited-environment clarification per §11.4.209 reviewer MINOR-4 (POSIX sh with only rung 5 SATISFIES the anchor IFF the runtime assertion GENUINELY CATCHES the invariant AND is paired with its §1.1 meta-test mutation per §11.4.107(10) self-validation). Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-241-PROPAGATION` (literal `11.4.241`) + recommended gate `CM-ILLEGAL-STATE-LADDER-PREFERENCE` + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227).

Canonical authority: constitution submodule `Constitution.md` §11.4.241. Non-compliance is a release blocker. No escape hatch — no `--runtime-assert-suffices`, `--skip-type-preference`, `--dynamic-language-exempts-all-rungs`, `--property-test-in-lieu-of-type`, `--decorative-type-annotation-OK` flag.

---

§ . . — Byte-identical-fork prohibition + role-as-data-pack (research-derived, - -)

Full anchor per Phase 3 landing brief §11.4.251. When project needs would be met by TWO NEAR-IDENTICAL CODE COPIES differing only in configuration/data/role-specific parameters (prod + staging with hardcoded hostnames; reader + writer with different flag values; web + mobile with different feature flags), copies MUST NOT be maintained as parallel forks. Instead ONE code copy retained + role-specific values extracted into DATA PACK (role-config file / env var set / capability descriptor / feature-flag bundle) consumed by the one code to instantiate as either role. Extraction line 6490 [offset un-resolvable per the §11.4.239 CITATION NOTE; **MODULE ADDED 2026-08-20** — resolves to **module 35** taxonomy row 8 "Byte-identical-fork", verified locators `module_35:127` (row header), `module_35:130` (the SonarQube-Factory case), `module_35:132` (the "roles become data-only packs you reference, never repos you fork" remedy)]: byte-identical-fork taxonomy row + SonarQube-Factory case (`Application/main.kt` byte-for-byte the caching-proxy launcher, still in `package net.milosvasic.factory.proxy.caching.application` — "no SonarQube code"; fix = role-real acceptance test, "roles become data-only packs you reference, never repos you fork"). DETECTION: reviewer or mechanical scan finds byte-identical fork when `diff` shows every difference is a config value (hostname/port/flag/label/env/credential-ref) NOT behavioral, common git-history ancestor, > 90% byte identity; per §11.4.35 threshold-DATA. EXTRACTION: (1) IDENTIFY configuration surface; (2) EXTRACT to data pack per role; (3) REFACTOR single code copy to consume at instantiation (constructor injection / factory / config-loader-at-startup); (4) DELETE one fork; (5) VERIFY per §11.4.5 both role instantiations produce same observable behavior their forks previously produced (§11.4.243 characterization before refactor); (6) commit extraction per §11.4.124 with git-history evidence. LEGITIMATE FORKS: temporary migration (tracked §11.4.197 end date), experimental / A-B (bounded end date), truly divergent behavior (copies differ in algorithm/protocol/data-model not just config). Composes §11.4.5 / §11.4.6 / §11.4.28 (decoupling — role-configurable code shared across consumers) / §11.4.35 (data-pack location) / §11.4.124 / §11.4.177 / §11.4.194 / §11.4.197 / §11.4.243 / §1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-251-PROPAGATION` (literal `11.4.251`) + recommended gate `CM-NO-BYTE-IDENTICAL-FORK` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.251. Non-compliance is a release blocker. No escape hatch — no `--fork-for-role-OK`, `--skip-data-pack-extraction`, `--copy-and-modify-config`, `--duplicate-code-per-environment`, `--near-identical-forks-are-fine` flag.

---

§ . . — Review-round budget: every independently-reviewed work item converges within a declared budget of at most rounds (- , consumer-declared), by front-loading ground truth, class-exhaustive enumeration, real-path tests and a declared threat boundary, with a binding methodology change at round (User mandate, - -)

Forensic anchor — verbatim operator mandate (2026-10-05, `CRITICAL`): "Make sure we never ever get to such extremely high never ending review rounds count! Determine exact causes of issues we are facing with deep planning and exhaustive analisys so all work ALWAYS gets done with 5 to 7 reviews max! This MUST BE mandatory part of the constitution!!! Reasonable reviews rounds count with fully deterministically validated and verified working results produced!"

Forensic FACTS (genericised; full citations in the consuming project's own continuation record). One reference consuming project ran four independently-reviewed work items to 7, 10+, 30 and 39+ rounds in one week. Reading every round's recorded findings showed the count was driven by five recurring, avoidable mechanisms, not by an unusually hard problem: **(1) wrong-model ground truth** — a build-configuration fix and its OWN verifying gate were both built on a simplified mental model of an external build system's inheritance mechanism (values assumed to flow through a plain include chain, when the real tool clears the variable before a node's own include and merges ancestor values in afterwards); the gate even ran the real tool, but on a hand-simplified fixture, so it reported GREEN on a fix that could never work; it was caught only when a reviewer read the tool's own source and rebuilt the fixture on the tool's REAL composition path, after which the fix was rebuilt on the real path and landed in a single remediation round; **(2) instance-at-a-time closure of a known class** — a security gate was hardened against command-shadowing one command name per round (each of rounds 36,

37 and 38 found further unhardened names) after round 35 had explicitly considered and declined a mechanical whole-file enumeration because no decision-path classifier existed; in a second item a "close the whole class" instruction was met by fixing a syntactic proxy token rather than the class, and the same defect family returned in four consecutive rounds; **(3) fixer-authored tests that never drive the real code** — at rounds 25, 26, 27, 37 and again 38 a reviewer showed the newest regression test re-implemented, text-extracted, or never drove a verdict through the logic it claimed to guard, so the reviewer's own mutations of the real code survived the whole suite; **(4) non-binding escalation signals** — the heuristic-tower anchor (§11.4.250) was invoked about thirty times across these items (once, at round 5, with the explicit prediction "point fixes alone are likely to produce a Round 6 of the same kind") and the stall anchor (§11.4.267) had no declared retry bound anywhere in the project, so neither ever stopped the loop; a reviewer-recommended structural fix (re-executing the gate at its call seam in a scrubbed environment) was deferred in two successive rounds while in-file point hardening continued; **(5) scope drift** — a review loop absorbed an out-of-scope tangent (a separate, unrelated hardening effort surfaced mid-review) and the round counter kept accruing against the original item for the whole of that tangent, inflating its own count substantially before the tangent was explicitly separated out. The same record also contains genuinely new findings that legitimately needed another round (a shell-parser asymmetry that made a core test operator shadowable; a sentinel variable an attacker could pre-set) — iteration as such is not the defect. The defect is rounds spent re-discovering what one front-loaded investigation would have found, or accruing against an item the item never actually owned.

What this is NOT (the §11.4.227 boundary, stated first). §11.4.134 governs the review loop's GO bar (zero findings, zero warnings) and its obligation to keep re-running until that bar is met. §11.4.267 governs the shared attempt record, converge-on-evidence, and an undeclared stall bound `N`. §11.4.250 governs the heuristic-tower SIGNAL that a primitive is defective. None of the three bounds the review loop's LENGTH or binds a concrete action to a round NUMBER. §11.4.276 adds exactly that — clause (A) the ceiling, clause (E) the round-3 checkpoint and the ceiling escalation. Clauses (B)–(D) add no new substantive obligation beyond fixing WHEN existing duties must be discharged: (B) binds §11.4.6 / §11.4.99 / §11.4.102 phase 1 / §11.4.199 / §11.4.226 to a recorded artifact at a fixed point

before the first fix; (C) binds §11.4.250 / §11.4.201(7)(b) to the round that first meets a nameable class; (D) binds §11.4.224(A) / §11.4.194(6)(d) / §11.4.115(F) / §11.4.235(D) to every remediation round's own regression set.

The mandate (ALL hold):

(A) DECLARED ROUND BUDGET, HARD CEILING 7. Every work item that passes through the §11.4.125 / §11.4.134 / §11.4.142 / §11.4.194 review loop carries a round budget `R_max`, declared by the consuming project as DATA (§11.4.35) in the closed range 5 ≤ `R_max` ≤ 7; an absent declaration resolves to 5 (the conservative value — the escalation in (E) fires earliest; §11.4.101). The ceiling is a constitutional literal because the operator supplied it verbatim (§11.4.246's operator-decided fleet minimum is the same pattern); the 5–7 range is the operator-mandated ceiling recorded with its own verbatim mandate above, not an invented `N` attributed to an external source (§11.4.267(E)) — the consumer's declared `R_max` within that ceiling IS its §11.4.267 `N` for the review loop, and remains consumer DATA. A round = one independent review verdict on the item, whether on uncommitted or already-pushed work, counted from the first review and recorded with the item's review record; a review that never received its brief, or crashed without a verdict, is recorded but not counted. Splitting, renaming, or re-filing the item does NOT reset the count unless (E)(3) applies. A tracked item created from a finding raised during a budgeted item's review inherits that item's consumed round count — the exception is a finding outside a scope/threat boundary declared before round 1 under (E)(2), which starts a fresh budget; item identity for recurrences follows §11.4.214 (link, never mint).

(B) GROUND TRUTH BEFORE THE FIRST FIX. Before the first fix attempt — not after a reviewer finds the model wrong — for any change or finding whose correctness depends on the behaviour of a system the author does not own (build tool, compiler, interpreter or shell semantics, kernel, runtime, third-party library, protocol), the author MUST record a ground-truth probe: (1) the authoritative source of the behaviour (the tool's own source code or normative specification, cited by file and line — not a summary, not memory); (2) a minimal fixture executed on the REAL tool through its REAL composition path (the actual inheritance, sourcing, linking or dispatch mechanism the product uses — a hand-simplified approximation of that path does not count); (3) the recorded behavioural model the fix relies on. The fix's own verifying gate or test MUST exercise the same real path the probe

established; a gate verified on a simplified model is wrong-layer evidence (§11.4.226) and cannot support a GO. Composes §11.4.6 / §11.4.99 / §11.4.102 phase 1 / §11.4.199 — it binds them to a recorded artifact at a fixed point in time.

(C) CLASS-EXHAUSTIVE ENUMERATION ON FIRST INSTANCE. When a finding is an instance of a class (bypass via name resolution, an unguarded external command, a masked character, a mutation-surviving test, a missed parse case), the round that fixes it MUST (1) name the class; (2) enumerate every member across the item's whole scope with a mechanical instrument whose null result is control-needle-proven (§11.4.201(7)(b)), and record that inventory; (3) close every member, or scope a member out explicitly with a reason and a tracked item (§11.4.197). Fixing only the reported instance when the class is nameable is a §11.4.276(C) violation. When the enumeration needs a tool that does not yet exist (for example a classifier separating decision-path calls from display-only calls), building that tool IS the remediation work — "no classifier yet" is not a reason to fall back to instance-by-instance fixes. A claim that a class is "closed" MUST cite the inventory artifact; a fix that closes a syntactic proxy instead of the class is an unclosed finding (§11.4.250).

(D) REAL-PATH TESTS AND REVIEWER MUTATIONS ADOPTED VERBATIM. Every regression test added in a remediation round MUST drive the real shipped artifact through its real invocation path (§11.4.224(A)) — a test that re-implements the guarded logic inline, or runs extracted source text outside the artifact's own process, guards nothing and is a `source-defect` finding under §11.4.235(D). Every mutation a reviewer reported as surviving is added to the item's mutation set verbatim and must FAIL the suite before the next review is requested (composes §11.4.194(6)(d) / §11.4.115(F)). The fixer runs the item's full regression set and the reviewer's previous attacks against the fix before requesting review; the dispatch seam REFUSES to start a review when that record is absent (never counted as a round because it is never dispatched). A review that WAS actually dispatched and returned a verdict always counts as a round, regardless of whether its triggering record was later found deficient — the record requirement gates DISPATCH, it is never grounds to retroactively erase a verdict that already issued.

(E) BINDING METHODOLOGY CHANGE AT ROUND 3, AND THE CEILING ESCALATION. (1) At round 3, and at every later round, the round's fixer MUST first record a convergence assessment classifying every finding so far as new-class / same-class-as-earlier / regression-introduced-by-a-fix / wrong-layer-evidence. If two

or more findings share a defect class named under (C) (not merely a shared §11.4.235(D) finding layer) with at least one non-MINOR finding in that shared class, any finding reappeared after being reported fixed, or any finding was wrong-layer evidence, the next round MUST be a structural round: a fresh fixer agent instance that did not author rounds 1–2 (the reviewer remains structurally separate throughout per §11.4.240), re-deriving (B) from first principles, re-running the (C) enumeration over the whole scope, and replacing the approach rather than adding a further layer (§11.4.250). A structural fix recommended by a reviewer may not be deferred past this point without a recorded operator decision (§11.4.66). (2) Every review declares, before round 1, its scope and acceptance boundary (and, for adversarial/security reviews, the threat model — which actor capabilities are in scope, and the seam at which the defence lives) so that a zero-finding GO under §11.4.134 is reachable; findings outside the declared boundary are tracked items, not loop-extending findings. The boundary is fixed at round 1 and changes only by recorded operator decision — narrowing it mid-loop to reach GO is a §11.4.6 fabrication. (3) If round `R_max` returns anything but GO, the item MUST NOT run round `R_max` +1 under the same approach. It stops and goes to a recorded operator decision (§11.4.66) carrying: the finding classification, the evidence of which of (B)–(D) was skipped or failed, and the options (re-scope into smaller slices per §11.4.148; replace the approach with the structural design as a new item; or accept residual LOW findings only through a §11.4.271 waiver). Only a recorded operator decision opens a new budget. The escalated item is parked per §11.4.101 / §11.4.267(F) — it parks only that work unit; the autonomous loop (§11.4.126) continues all other work, and the pending decision is surfaced in every §11.4.97 progress report until resolved. Every breach is also recorded as a process escape and audited for which of (B)–(D) would have prevented it (the §11.4.238(C) pattern applied to the review loop).

(F) NOTHING HERE LOWERS THE BAR. The budget bounds the NUMBER of rounds; it never lowers what a round must establish. §11.4.134's zero-finding, zero-warning GO bar is unchanged; its re-run obligation is bounded PER APPROACH by this anchor's `R_max`, after which (E)(3) governs rather than a further same-approach round. §11.4.209's review substrate, §11.4.235(D)'s finding-layer classification and §11.4.262's machine-created evidence are likewise unchanged. Reaching round `R_max` without GO never produces a GO, a "good enough", or a silent close: the item stays open until a recorded operator decision under (E)(3).

Honest boundary (§11.4.6). This anchor does not promise that every item converges within `R_max` rounds. Some problems are hard, and a determined adversarial reviewer can keep finding real defects in an in-process defence whose threat boundary is wider than its design. What the anchor removes is the class of round inflation the forensic record shows to be avoidable: rounds spent discovering a wrong external-system model, one member of an already-known class, a test that never drove the real code, a stall signal nobody was required to act on, or an unrelated tangent silently accruing against the wrong item. When a problem is genuinely hard, (E)(3) turns an open-ended loop into a bounded, evidence-backed operator decision. The cap counts rounds, not effort; it does not license a shallower review to stay under it. The operator's own wording ("ALWAYS gets done with 5 to 7 reviews max") is read here as a ceiling with a mandatory escalation at that ceiling, not as a guarantee every item physically closes within it — this reading is recorded explicitly and was surfaced to the operator at landing per §11.4.66.

Classification: universal (§11.4.17) — no hardware, vendor or project literal; the consuming project supplies `R_max` (5–7), its review-record location, and its probe and inventory artifact paths as DATA per §11.4.35.

Composes §11.4.6 (no guessing — the ground-truth probe is its recorded form) / §11.4.66 (operator decision at the ceiling and for boundary changes) / §11.4.101 (conservative default `R_max` = 5) / §11.4.102 (systematic-debugging phase 1 bound to round 1) / §11.4.125 / §11.4.134 (the loop this budget bounds; its GO bar is unchanged) / §11.4.142 / §11.4.148 (re-scope by slicing) / §11.4.194(6)(d) (reviewer-authored mutations) / §11.4.197 / §11.4.201(7)(b) (control-needled enumeration) / §11.4.209 (review substrate, unchanged) / §11.4.214 (recurrence linking for spun-out findings) / §11.4.224(A) (real invocation path) / §11.4.226 (wrong-layer evidence) / §11.4.235(D) (finding layer) / §11.4.238(C) (escape audit pattern) / §11.4.240 (fresh actor / structural reviewer separation) / §11.4.250 (heuristic tower, made binding at round 3) / §11.4.267 (this anchor supplies its `N` for the review loop) / §11.4.271 (waiver for residual LOW findings) / §1.1.

Propagation gate `CM-COVENANT-114-276-PROPAGATION` (literal `11.4.276` as a block-start, exactly once per governance file, lockstep content-hash equality across the mirror set per §11.4.227(B)) + recommended mechanism gates:

- `CM-REVIEW-ROUND-BUDGET` — every reviewed item has a declared `R_max` in 5–7 (or the default 5) and a round count from its review record; a round `R_max` +1 dispatched with no recorded operator decision FAILs; an item closed without GO FAILs.
- `CM-REVIEW-ROUND3-CONVERGENCE-ASSESSMENT` — every round ≥ 3 carries a convergence classification; a non-structural round following an assessment that triggered (E)(1) FAILs; an adversarial review with no pre-round-1 threat boundary FAILs.
- `CM-FIX-GROUND-TRUTH-AND-CLASS-INVENTORY` — a fix touching a declared external-system surface with no ground-truth probe record, or a class finding fixed with no inventory artifact, FAILs.

Paired §1.1 mutations: dispatch a round 8 with no operator-decision record → `CM-REVIEW-ROUND-BUDGET` MUST FAIL; delete the round-3 classification, or follow a same-class trigger with a point-fix round → the round-3 gate MUST FAIL; strip the probe record from a build-tool fix, or mark a class closed with no inventory → the third gate MUST FAIL; strip the literal → propagation FAILs. Golden-FALSE per §11.4.201(1): an item reaching GO at round 4 with a round-3 assessment showing only new-class findings, a recorded probe, and an inventory for its one class finding MUST NOT fire any gate. The propagation gate `CM-COVENANT-114-276-PROPAGATION` is implemented in the same change (`constitution/scripts/gates/cm_covenant_114_276_propagation.sh` + its paired mutation test); the three recommended mechanism gates are registered as tracked deferrals (OWED-GATE-111..113) in `constitution/scripts/gates/gate_ledger_deferrals.tsv` — their gate-code is a separate work item, NOT claimed shipped (§11.4.6 / §11.4.227).

Canonical authority: constitution submodule `Constitution.md` §11.4.276.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--unbounded-review-rounds`, `--fix-instance-not-class`, `--defer-ground-truth-probe`, `--narrow-threat-model-to-reach-go`, `--reset-round-count-on-refile`, `--defer-structural-fix-past-round-3` flag.